

Universidad Internacional del Ecuador

Ingeniería en Sistemas de la Información

Programación Orientada a Objetos

Proyecto Integrador - Fase 1-2

Joselyn Paulet Laguna Puetate

Jose Emiliano Velasco Sosa

Darwin Oswaldo Román Aguirre

5 de octubre de 2026

Índice

Descripción del problema.....	3
Justificación del sistema seleccionado.....	4
Objetivos.....	5
Objetivo General.....	5
Objetivos específicos.....	5
Alcance del Proyecto.....	6
Restricciones técnicas y operativas.....	7
Exclusiones del Sistema.....	7
Requisitos funcionales.....	8
Requisitos no funcionales.....	11
Actores y usuarios.....	12
Casos de uso.....	14
Descripciones detalladas de casos de uso principales.....	18
Caso de uso 1: Iniciar sesión en el sistema.....	18
Caso de uso 2: Enviar y recibir mensajes de texto en red.....	19
Caso de uso 3: Visualización del Historial Básico y Cierre de Sesión Seguro.....	20
Matriz de trazabilidad inicial.....	21
Conclusión.....	24

Etapas 1. Planeación del software

Descripción del problema

Actualmente la empresa presenta dificultades en la comunicación interna entre los colaboradores que trabajan en sus dos sedes y gran parte de la información operativa debe transmitirse mediante llamadas, mensajes personales, correos electrónicos u otros medios que no están centralizados lo que puede provocar retrasos, pérdida de información y dificultades para mantener una comunicación ordenada entre los equipos.

Este problema afecta principalmente a los empleados que necesitan intercambiar información de manera constante entre una sede y otra para coordinar actividades, realizar consultas, comunicar novedades o solicitar apoyo y cuando la comunicación depende de medios diferentes ya que resulta más difícil conservar un registro de las conversaciones y garantizar que la información llegue oportunamente a las personas correspondientes

Entre las principales consecuencias se encuentran la demora en la respuesta entre colaboradores, la duplicación de mensajes, la pérdida de información importante y la dificultad para revisar conversaciones anteriores. También puede existir confusión cuando una misma información es enviada por diferentes canales o cuando los empleados no saben cuál es el medio oficial para comunicarse internamente

El problema se limita específicamente a la comunicación mediante mensajería instantánea entre usuarios pertenecientes a las dos sedes de la organización y la propuesta no busca reemplazar todos los canales de comunicación de la empresa sino proporcionar una herramienta interna que permita a los colaboradores intercambiar mensajes de manera organizada y centralizada.

Una aplicación de chat desarrollada en Go puede contribuir a solucionar este problema mediante una arquitectura cliente-servidor. Los usuarios de ambas sedes podrán utilizar un programa cliente para conectarse a un servidor central, autenticarse y enviar o recibir mensajes con otros colaboradores conectados al sistema. Go es adecuado para este tipo de aplicación debido a que permite gestionar múltiples conexiones de manera eficiente mediante concurrencia y esto resulta especialmente útil en un sistema de mensajería, ya que varios usuarios pueden estar conectados y enviando mensajes al mismo tiempo.

De esta manera la aplicación permitirá mejorar la comunicación entre las dos sedes, reducir la dependencia de canales informales y disponer de un medio común para el intercambio de mensajes entre los colaboradores de la organización.

Justificación del sistema seleccionado

Para este proyecto se eligió desarrollar un chat empresarial porque la comunicación entre las dos sedes es importante para el trabajo diario de la empresa y los colaboradores ya que necesitan realizar consultas, coordinar actividades y compartir información con personas que se encuentran en la otra sede y por esta razón es útil contar con un medio de comunicación interno que sea sencillo y centralizado y así los empleados podrán comunicarse de forma directa sin depender siempre de llamadas, correos electrónicos o aplicaciones personales.

Los principales beneficiarios serán los trabajadores de las dos sedes ya que el sistema permitirá iniciar sesión y ver qué usuarios están conectados, enviar mensajes, recibir mensajes y revisar conversaciones anteriores además la aplicación también guardará

información básica de los usuarios como el nombre, las credenciales de acceso, el estado de conexión, el remitente, el destinatario y la fecha de cada mensaje.

La arquitectura cliente-servidor es importante porque permitirá que todos los usuarios se conecten a un mismo servidor incluso si se encuentran en sedes diferentes. El servidor será el punto central de comunicación. Su función será recibir los mensajes y enviarlos al usuario correcto. Por ejemplo, un trabajador de una sede podrá enviar un mensaje a otro empleado ubicado en la segunda sede y el servidor se encargará de entregarlo.

También se eligió este sistema porque permite aplicar varios conocimientos de programación en Go y durante el desarrollo se trabajará con usuarios, conexiones de red, validaciones, manejo de datos y comunicación entre varios clientes y por estas razones el chat empresarial es una opción adecuada para el proyecto ya que permite atender una necesidad real de la empresa y aplicar de forma práctica los conocimientos aprendidos en la materia.

Objetivos

Objetivo General

Desarrollar una aplicación de chat corporativo en arquitectura cliente/servidor, en el lenguaje Go, que permita la interacción concurrente y segura entre usuarios de dos unidades organizativas distintas, con verificación de identidad y sobre una infraestructura de red segmentada.

Objetivos específicos

Funcionalidad: Implementar las funciones principales de mensajería para enviar y recibir mensajes de texto en ambas direcciones, así como guardar y recuperar el historial básico de conversaciones de los usuarios conectados.

Integración: Integrar la aplicación cliente-servidor desarrollada en Go con el servicio externo de autenticación mediante FastAPI para gestionar de forma fluida la validación de identidad en el acceso a la plataforma.

Seguridad: Controlar el acceso a los recursos de la aplicación mediante la verificación y protección de credenciales con hashing seguro y sesiones válidas que restrinjan el ingreso a usuarios no autorizados.

Mantenibilidad: Organizar el código fuente del software aplicando conceptos de Programación Orientada a Objetos, como diseño modular, encapsulación y manejo estructurado de errores, para garantizar un sistema escalable y cooperativo.

Validación: Verificar la concurrencia y la funcionalidad de conectividad del sistema para validar su funcionamiento general a través de una infraestructura de red segmentada por VLAN y enrutamiento OSPF.

Alcance del Proyecto

El proyecto abarca el diseño e implementación de un sistema de comunicación corporativa que combina software y redes. Del lado del software, incluye el desarrollo de la lógica de un chat cliente-servidor implementado en el lenguaje Go (Golang) y un servicio de autenticación centralizado desarrollado en el framework FastAPI. Del lado de la infraestructura, comprende el diseño y la simulación completa de una topología de red que interconecta dos sedes utilizando direccionamiento IP, segmentación mediante VLANs, enlaces troncales (Trunk), enrutamiento inter-VLAN con la técnica Router-on-a-Stick, enrutamiento dinámico mediante el protocolo OSPF y redundancia en capa 2 con el protocolo RSTP. Adicionalmente, incluye la fase de recolección de métricas técnicas y su respectiva evaluación cuantitativa del rendimiento de la red y la aplicación.

Restricciones técnicas y operativas

El desarrollo del software principal del chat debe realizarse en el lenguaje de programación Go.

El servicio de autenticación y seguridad de acceso debe implementarse utilizando el framework FastAPI.

La infraestructura de red y sus componentes de conectividad y seguridad deben simularse de forma obligatoria en herramientas de simulación de redes compatibles y aprobadas, específicamente Cisco Packet Tracer.

El desarrollo debe ejecutarse dentro de los plazos establecidos, ajustándose a los recursos tecnológicos locales disponibles por el equipo de trabajo.

Exclusiones del Sistema

- **Despliegue en la nube comercial:** Queda excluido el despliegue del sistema en servidores o plataformas comerciales de pago en la nube (AWS, Azure o Google Cloud); la validación del funcionamiento se realizará exclusivamente mediante entornos locales, contenedores y simulaciones controladas en la infraestructura de red de las sedes.
- **Multimedia en tiempo real y transmisión de voz/video:** Se excluye el soporte para transmisión de llamadas de voz, videollamadas, notas de voz o streaming de video; el alcance del sistema está limitado estrictamente a la mensajería de texto sincronizada y estructurada para entornos corporativos.
- **Transferencia y almacenamiento de archivos pesados:** Queda fuera del alcance el intercambio de archivos adjuntos de gran tamaño (como documentos PDF extensos, imágenes de alta resolución o ejecutables

binarios), limitando la carga útil (payload) del chat a cadenas de texto e historial de mensajes ligeros.

- **Interfaces gráficas avanzadas de usuario (GUI de Escritorio / Web**

nativa): Se excluye el desarrollo de aplicaciones móviles nativas

(Android/iOS) o interfaces web complejas basadas en frameworks pesados de frontend; la interacción con el cliente de chat en Go se enfocará en interfaces de consola avanzadas o terminales de texto interactivas para priorizar la lógica de sockets y concurrencia.

- **Sistemas de enrutamiento externo o pasarelas de mensajería pública:**

Queda excluido cualquier tipo de integración con pasarelas de mensajería externa, redes sociales o APIs de terceros (como WhatsApp Business API, Telegram o SMS), operando el chat de manera estrictamente interna y aislada dentro de la red corporativa simulada.

Requisitos funcionales

Código	Requisito (funcional)	Criterio de aceptación en base al proyecto
RF-SW-01	Aplicación cliente-servidor en Go.	Un cliente ubicado en una sede consume una funcionalidad ejecutada por el servidor de chat en Go y recibe una respuesta válida.
RF-SW-02	Operaciones principales del sistema seleccionado.	Las funciones esenciales del chat se completan sin errores críticos y persisten o transmiten los mensajes de texto requeridos.

RF-SW-03	Integración con autenticación.	El acceso a los recursos protegidos del chat exige una sesión válida emitida por el servicio de autenticación en FastAPI.
RF-NET-01	Dos sedes interconectadas.	Existe conectividad IP entre redes autorizadas de ambas sedes simuladas en Packet Tracer.
RF-NET-02	VLAN, trunk y Router-on-a-Stick.	Las VLAN se encuentran activas, los enlaces transportan etiquetas 802.1Q y el enrutamiento inter-VLAN funciona según las políticas.
RF-NET-03	OSPF entre sedes.	Los routers intercambian rutas dinámicamente y las tablas reflejan las redes remotas de las sucursales.
RF-NET-04	EtherChannel y RSTP.	Los enlaces agregados operan correctamente y la topología evita bucles de capa 2 mediante el protocolo RSTP.
RF-SEC-01	Registro, inicio de sesión y autorización.	Credenciales válidas permiten acceso; credenciales inválidas son rechazadas y los recursos restringidos permanecen protegidos.
RF-SEC-02	Protección de credenciales.	Las contraseñas se almacenan mediante hash seguro. AES se utiliza solo para datos sensibles que requieran recuperación reversible.

RF-SEC-03	Firewall y segmentación segura.	Las comunicaciones no autorizadas entre redes o departamentos son bloqueadas y las reglas se documentan.
RF-EST-01	Recolección de datos.	Existe un instrumento o procedimiento repetible para obtener métricas técnicas (como latencia del chat) o datos de percepción.
RF-EST-02	Análisis estadístico.	Se presentan tablas, gráficos, medidas descriptivas y una interpretación vinculada con los requisitos de rendimiento.
RF-TS-01	Análisis socio-tecnológico.	Se identifican actores, beneficios, riesgos de privacidad, accesibilidad y recomendaciones éticas del uso del chat corporativo.
RF-INT-01	Integración completa.	La aplicación de chat, la autenticación FastAPI y la red de las sedes funcionan conjuntamente durante una demostración reproducible.

Requisitos no funcionales

Código	Requisito (No Funcional)	Descripción aplicada al proyecto
RNF-01	Rendimiento	Los tiempos de respuesta del chat y la latencia en la red serán medidos y comparados con un umbral definido por el equipo.
RNF-02	Disponibilidad	Los servicios de red y el servidor de chat permanecerán operativos durante las pruebas planificadas y se documentarán interrupciones.
RNF-03	Seguridad	Se aplicará mínimo privilegio, validación de entradas en Go, protección de credenciales y reglas de firewall en los equipos de red.
RNF-04	Mantenibilidad	Código modular en Go, configuraciones documentadas, repositorio Git organizado y README actualizado en el gestor de código.
RNF-05	Usabilidad y accesibilidad	La interfaz del chat será comprensible, consistente y evaluada mediante criterios definidos para el usuario final.
RNF-06	Escalabilidad	La arquitectura de red y software permitirá incorporar nuevas VLANs, usuarios o servicios sin un rediseño completo.

		
RNF-07	Validación científica	Las pruebas serán repetibles y las conclusiones de rendimiento se sustentarán en evidencia cuantitativa obtenida de las mediciones.

Actores y usuarios

Actor	Descripción	Responsabilidades	Operaciones permitidas
Usuario Interno (Cliente)	Miembro u operador de la organización, físicamente situado en la Sede A o en la Sede B, que emplea la interfaz cliente de la aplicación para conectar con otros miembros de la empresa.	Usar credenciales válidas para identificarse en el sistema, proteger el acceso a su cuenta personal y utilizar la plataforma solamente para el intercambio de información corporativa autorizada.	1. Iniciar sesión en el sistema. 2. Enviar mensajes de texto a otros usuarios. 3. Recibir mensajes entrantes. 4. Visualizar el historial básico de la sesión activa. 5. Cerrar sesión de manera segura.

Servidor de Chat (Componente Go)	Software centralizado escrito en Go que actúa de núcleo de arquitectura cliente-servidor, intermediando, procesando y distribuyendo el tráfico de mensajes de la red.	Mantener el servicio en modo escucha de forma continua, gestionar la concurrencia de varios clientes sin que se degrade el servicio, enrutar cada mensaje a su destinatario correcto y asegurar la persistencia temporal de los datos.	1. Aceptar conexiones de red entrantes. 2. Delegar la validación de credenciales a la API. 3. Retransmitir paquetes de texto entre sockets de clientes. 4. Generar y estructurar el historial temporal.
Servicio de Autenticación (API FastAPI)	Sistema externo e independiente de la aplicación de mensajería para el control de acceso y proveedor de identidad de la red corporativa.	Procesar de forma confidencial las peticiones de acceso, comparar las credenciales recibidas con los hashes almacenados en su base de datos y emitir respuestas inequívocas de autorización o de denegación.	1. Recibir peticiones de inicio de sesión. 2. Verificar la integridad de las contraseñas (hashes). 3. Emitir estados de autorización de sesión. 4. Rechazar accesos inválidos.

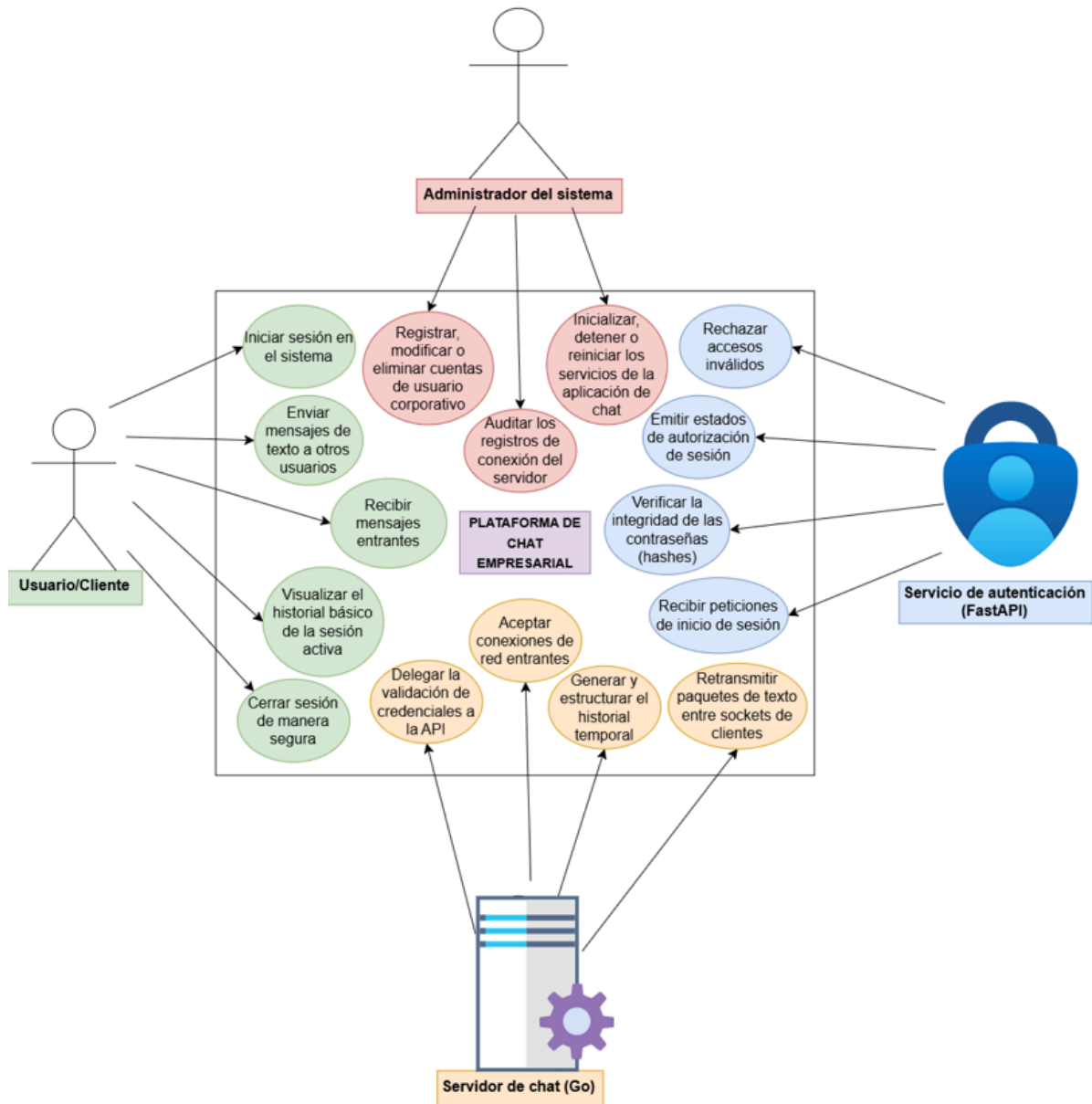
Administrador del Sistema / Red	Técnico especialista personal de la compañía encargado de la gestión de la infraestructura tecnológica, del despliegue de los servicios (Go y FastAPI) y de la configuración de la red de fondo.	Verificar que los servidores estén disponibles en las VLANs correspondientes, configurar las reglas del firewall para permitir el tráfico en los puertos correspondientes y gestionar el ciclo de vida de los usuarios en la base de datos.	1. Registrar, modificar o eliminar cuentas de usuario corporativo. 2. Auditar los registros de conexión del servidor. 3. Inicializar, detener o reiniciar los servicios de la aplicación de chat.

Casos de uso

Figura 1

Diagrama General del Sistema de Chat Empresarial

Diagrama 1: Funcionamiento General del Sistema

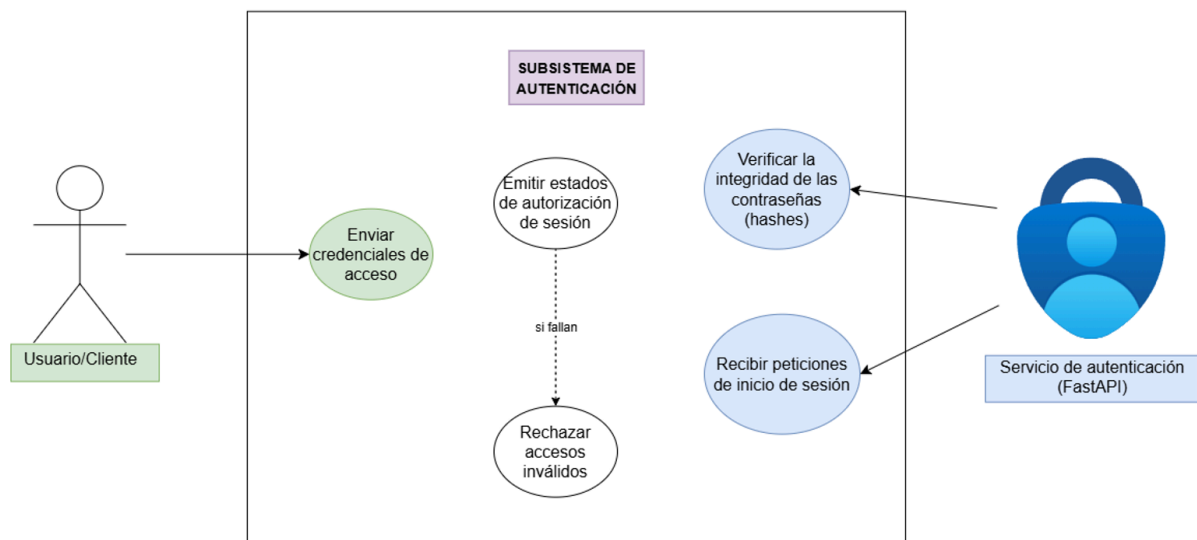


Nota. El diagrama ilustra la arquitectura general del sistema, representando a los cuatro actores principales (Usuario/Cliente, Administrador del sistema, Servidor de chat en Go y Servicio de autenticación FastAPI) interactuando con las operaciones globales de la plataforma, tales como el control de sesiones, la mensajería y la gestión administrativa.

Figura 2

Diagrama del Subsistema de Autenticación y Control de Acceso

Diagrama 2: Subsistema de Autenticación y Control de Acceso

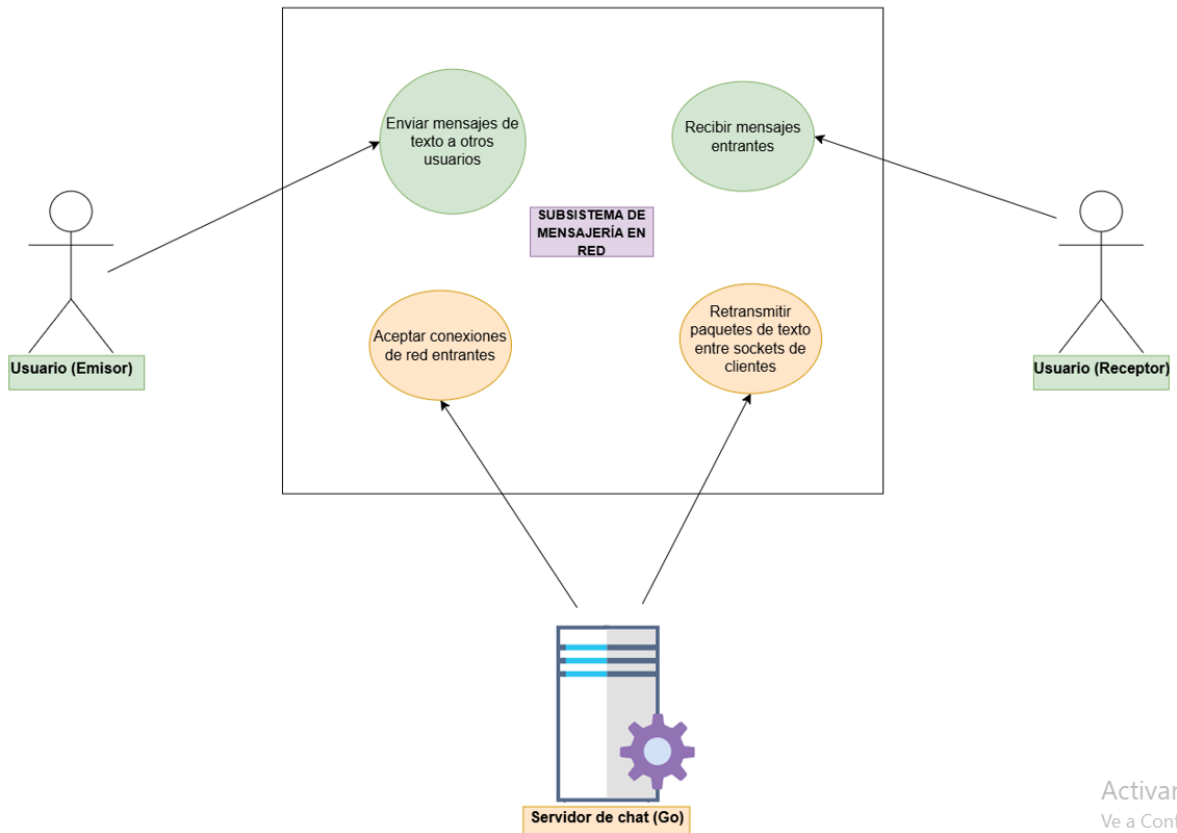


Nota. Este diagrama detalla el flujo de seguridad para el acceso a la plataforma. Muestra cómo el usuario envía sus credenciales al sistema y cómo el Servicio de Autenticación (FastAPI) verifica las contraseñas mediante hashes, emitiendo una sesión válida o activando una extensión condicional (si fallan) para rechazar accesos no autorizados.

Figura 3

Diagrama del Subsistema de Mensajería Concurrente (Cliente-Servidor en Go

Diagrama 3: Subsistema de Mensajería Concurrente (Cliente-Servidor en Go)



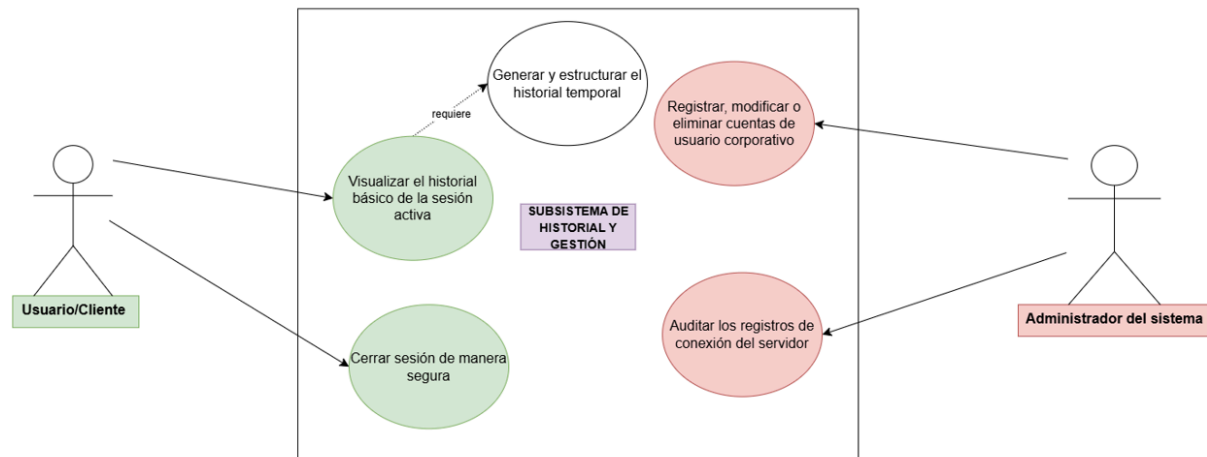
Activar
Ve a Conf

Nota. El diagrama modela la operación en red del chat corporativo, especificando la interacción entre el usuario emisor, el servidor central implementado en Go (encargado de aceptar conexiones de red y gestionar sockets de forma concurrente) y el usuario receptor.

Figura 4

Diagrama del Subsistema de Historial, Sesiones y Administración

Diagrama 4: Subsistema de Historial, Sesiones y Administración



Nota. Este diagrama muestra cómo el usuario puede ver el historial de sus mensajes recientes y cerrar su sesión de forma segura. Además, explica que para que el usuario pueda ver dicho historial, el sistema necesita ejecutar por detrás (de forma obligatoria) la función que genera los datos temporales en el servidor. Por otro lado, incluye las funciones exclusivas del administrador, como gestionar las cuentas de usuario y revisar los registros de conexión del servidor.

Descripciones detalladas de casos de uso principales

Caso de uso 1: Iniciar sesión en el sistema

- **Actor:** Usuario/Cliente, servicio de autenticación (FastAPI).
- **Descripción:** El usuario ingresa sus credenciales corporativas para autenticarse y acceder a la plataforma de chat.
- **Precondiciones:** El usuario debe estar registrado previamente en la base de datos de la empresa; el servicio FastAPI y el servidor de chat en Go deben estar en ejecución.
- **Postcondiciones:** Se emite un token/estado de sesión válido y se otorga acceso a la interfaz principal de mensajería.
- **Flujo principal:**
 1. El usuario abre el cliente de chat e introduce su usuario y contraseña.

2. El sistema envía las credenciales mediante una petición HTTP al microservicio de autenticación en FastAPI.
 3. FastAPI verifica la integridad de la contraseña mediante hashing seguro.
 4. El servicio valida la autenticidad y retorna un estado de autorización exitoso.
 5. El cliente de chat permite el ingreso del usuario a la interfaz.
- **Flujo Alternativo:** Paso 3.1 (Credenciales inválidas): Si la contraseña no coincide o el usuario no existe, FastAPI rechaza el acceso de forma inequívoca y el sistema muestra un mensaje de error impidiendo el ingreso.

Caso de uso 2: Enviar y recibir mensajes de texto en red

- **Actor:** Usuario emisor, servidor de Chat (Go), usuario receptor.
- **Descripción:** Transmisión concurrente de mensajes de texto entre colaboradores ubicados en diferentes sedes a través del servidor central.
- **Precondiciones:** Los usuarios deben tener una sesión activa validada; la red de las sedes debe mantener conectividad IP mediante OSPF.
- **Postcondiciones:** El mensaje es entregado en tiempo real en la interfaz del receptor y se almacena en el registro temporal del servidor.
- **Flujo Principal:**
 1. El usuario emisor redacta un mensaje de texto en su aplicación cliente y presiona enviar.
 2. La aplicación cliente transmite el paquete de texto mediante un socket hacia el servidor central en Go.
 3. El servidor en Go procesa la concurrencia, identifica al destinatario y retransmite el paquete al socket del usuario receptor.
 4. El usuario receptor visualiza el mensaje entrante en su pantalla.

- **Flujo Alternativo:** Paso 2.1 (Pérdida de enlace o desconexión): Si el servidor no detecta el socket activo del receptor, almacena temporalmente el mensaje en cola o notifica al emisor el fallo de entrega.

Caso de uso 3: Visualización del Historial Básico y Cierre de Sesión Seguro

- **Actor:** Usuario/Cliente, servidor de chat (Go).
- **Descripción:** Permite al usuario consultar el registro temporal de sus interacciones recientes durante la sesión activa y finalizar su conexión de manera ordenada.
- **Precondiciones:** El usuario debe encontrarse autenticado y con una conexión activa de socket establecida con el servidor en Go.
- **Postcondiciones:** Los registros temporales de la sesión se muestran en la interfaz y, al cerrar sesión, el socket se libera de forma segura en el servidor.
- **Flujo Principal:**
 1. El usuario selecciona la opción de visualizar el historial básico en su cliente.
 2. El cliente solicita al servidor en Go los paquetes de texto almacenados temporalmente en la memoria de la sesión.
 3. El servidor procesa la consulta y retorna el bloque de mensajes recientes al cliente.
 4. Posteriormente, al finalizar sus actividades, el usuario selecciona la opción de cerrar sesión.
 5. El cliente notifica al servidor, el cual desvincula el socket y borra el contexto temporal de la sesión activa.

- **Flujo Alternativo:** Paso 2.1 (Historial vacío): Si la sesión acaba de iniciar y no existen registros previos, el servidor responde con una estructura vacía y el cliente muestra una interfaz limpia sin interrumpir el flujo.

Matriz de trazabilidad inicial

Necesidad identificada	Objetivo relacionado	Código del requisito	Caso de uso	Criterio de aceptación
Falta de un canal oficial y seguro para la comunicación interna entre las dos sedes, evitando la dispersión de información en llamadas o correos no centralizados.	Controlar el acceso a los recursos de la aplicación mediante la verificación y protección de credenciales con hashing seguro y sesiones válidas que restrinjan el ingreso a usuarios no autorizados.	RF-SW-03 RF-SEC-01 RF-SEC-02 RNF-03	Iniciar sesión en el sistema / Rechazar accesos inválidos (Figura 2)	El usuario se identifica con credenciales válidas y el sistema valida la identidad a través de la API externa de FastAPI utilizando hashing seguro antes de permitir el acceso; los accesos erróneos son rechazados de forma inequívoca.

Demoras y necesidad de coordinar actividades y enviar novedades de manera constante entre los colaboradores de la Sede A y la Sede B.	Implementar las funciones principales de mensajería para enviar y recibir mensajes de texto en ambas direcciones, así como gestionar la interacción concurrente de los usuarios conectados.	RF-SW-01 RF-SW-02 RF-NET-01 RF-NET-03	Enviar mensajes de texto a otros usuarios / Recibir mensajes entrantes (Figura 3)	El servidor desarrollado en Go gestiona múltiples conexiones de red mediante concurrencia y sockets, asegurando que los paquetes de texto se retransmitan correctamente y sin pérdidas entre el emisor y el receptor.
---	---	--	---	---

Dificultad para revisar conversaciones anteriores o conservar un registro de los intercambios operativos recientes entre equipos.	Implementar el almacenamiento y recuperación del historial básico de conversaciones para los usuarios que se encuentran conectados en la plataforma.	RF-SW-02 RF-EST-01 RNF-01	Visualizar el historial básico de la sesión activa (Figura 4)	La interfaz del cliente permite visualizar de manera temporal el registro de mensajes estructurados y generados por el servidor durante la sesión activa del usuario.
Necesidad de gestión tecnológica y control sobre el ciclo de vida de las cuentas de los usuarios corporativos por parte del personal técnico.	Organizar el código fuente aplicando conceptos de Programación Orientada a Objetos y verificar la conectividad general a través de una infraestructura de red segmentada por VLAN y	RF-NET-02 RF-NET-04 RF-SEC-03 RNF-04 RNF-06	Registrar, modificar o eliminar cuentas de usuario corporativo / Auditar los registros de conexión del servidor (Figura 4)	El administrador del sistema puede gestionar las cuentas corporativas en la base de datos y auditar los registros de conexión del servidor asegurando la disponibilidad sobre las VLANs correspondientes.

Powered by Arizona State University	enrutamiento OSPF.			
---	---------------------------	--	--	--

Conclusión

Como conclusión, podemos decir que la aplicación de chat corporativo es totalmente viable y necesaria para resolver los problemas de comunicación y la dispersión de información que sufren las dos sedes de la empresa en el día a día. Existe una gran coherencia entre lo que diagnosticamos en el problema y los requisitos planteados, ya que cada necesidad de los trabajadores se conecta directamente con los objetivos y los casos de uso que diseñamos. Para la fase de diseño y desarrollo en Go, prevemos varios retos importantes: primero, lograr manejar de manera eficiente la concurrencia para que varios usuarios envíen y reciban mensajes al mismo tiempo sin que el servidor se sature; segundo, controlar correctamente la transmisión de datos a través de los sockets y hilos de ejecución (goroutines); y tercero, asegurar que los paquetes de texto se enruten sin errores entre las sedes y se integren bien con la seguridad de FastAPI. Por último, para la siguiente etapa tendremos que definir decisiones clave, como la estructura exacta para guardar el historial temporal de los mensajes, el manejo de las sesiones activas y las pruebas finales sobre la red segmentada.

Link del video:

<https://drive.google.com/file/d/1BgLwMJJoLMNJwATurPaoKMaBrOg6PJpEg/view?usp=drivesdk>

Link del repositorio:

<https://github.com/Joselyn-uide/Chat-Empresarial-PI>